

Guía de Actividades Práctico-Experimentales Nro. 002

1. Datos Generales

Estudiante	Isauro Michael Rivera Maldonado
Asignatura	Sistemas Distribuidos
Ciclo	6to
Unidad	Uno
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	2
Título de la Práctica	Arquitectura de Sistemas Distribuidos (comunicación cliente-servidor con archivos)
Nombre del Docente	José O. Guamán Q.
Fecha	06 de octubre de 2026 09 de octubre de 2026
Horario	Lunes 10h30 – 11h30 Viernes 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Comprender la arquitectura de sistemas distribuidos mediante el desarrollo independiente de sus componentes (cliente y servidor).

3. Materiales y reactivos:

Guía practica

Se requiere un sistema que permita crear un servidor y un cliente con un canal de comunicación (archivo de texto), se debe evidenciar que se esta llenado el archivo.

4. Equipos y herramientas

Hardware

- Computador

Software

- Lenguaje de programación
- Ide de Programación
- Archivo de Texto

5. Procedimiento / Metodología

Parte 1: Desarrollo del SERVIDOR (1 hora 20 minutos)

Este módulo se desarrolla de forma independiente

Actividad

1. Crear un archivo:
servidor.py
2. Crear un archivo auxiliar:
entrada_servidor.txt
3. Implementar un programa que:
 - Revise periódicamente el archivo entrada_servidor.txt.
 - Detecte si existe contenido nuevo.
 - Procese el mensaje:
 - Ejemplo: convertir texto a mayúsculas o invertirlo.
 - Escriba el resultado en: salida_servidor.txt
4. Condiciones:
 - Debe ejecutarse continuamente (simulación de servicio activo).
 - No debe reprocesar el mismo mensaje.
 - Debe manejar archivos vacíos.

◆ Parte 2: Desarrollo del CLIENTE (1 hora)

Se desarrolla sin necesidad de ver el código del servidor

Actividad

1. Crear archivo:
cliente.py
2. Implementar un programa que:
 - Solicite un mensaje al usuario.
 - Escriba el mensaje en: entrada_servidor.txt
 - Espere un tiempo (simulando latencia).
 - Lea la respuesta desde: salida_servidor.txt
 - Muestre el resultado en pantalla.
3. Condiciones:
 - No debe asumir cómo funciona el servidor internamente.
 - Solo debe conocer los archivos de entrada y salida.

◆ Parte 3: Integración del sistema (25 minutos)

1. Ejecutar primero el servidor.
2. Ejecutar el cliente.
3. Enviar varios mensajes.
4. Verificar:
 - Flujo correcto
 - Respuesta adecuada

NOTA: los archivos de comunicación deben estar en una sola carpeta fuera de ambos proyectos

6. Resultados esperados:

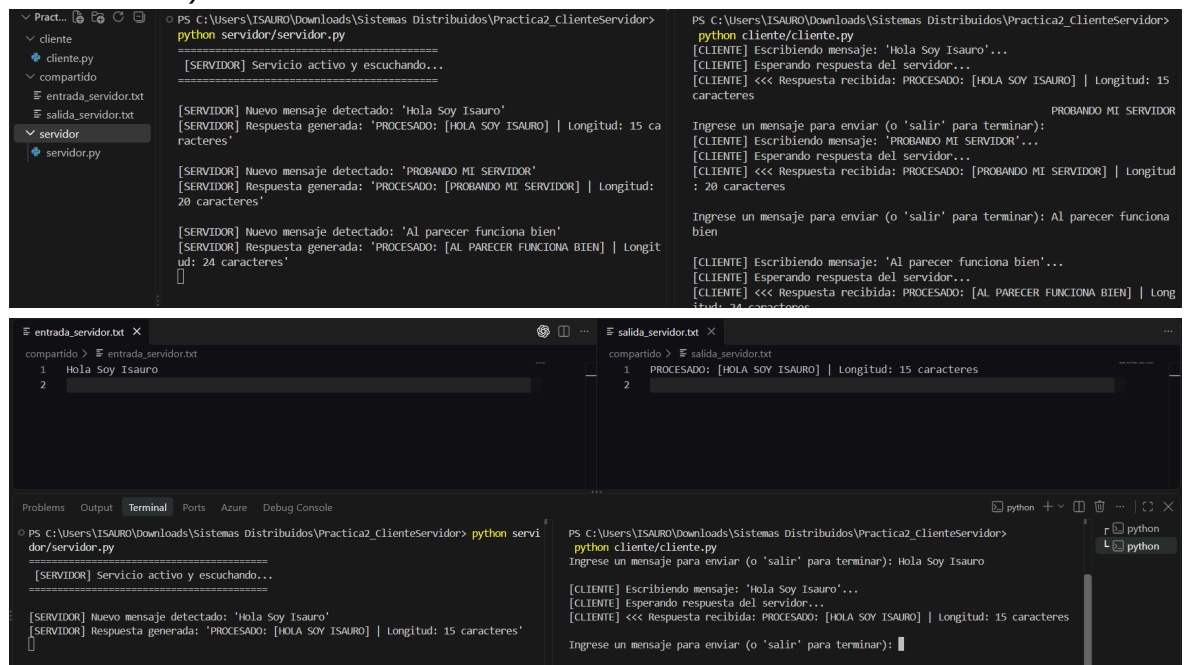
Al finalizar la práctica, el estudiante entregará:

- Código fuente de servidor.py y cliente.py (repositorio Git).

Repositorio Git:

<https://github.com/IsaKaiser04/SistemasDistribuidos.git>

- Capturas de pantalla que evidencien: (a) el archivo entrada_servidor.txt siendo llenado por el cliente, (b) salida_servidor.txt con el resultado procesado, (c) al menos 3 mensajes procesados correctamente.
- Respuestas a las preguntas de control en el reporte de práctica. Con ello demostrará: el desacoplamiento entre cliente y servidor, la comunicación indirecta por medio compartido, y las limitaciones del modelo (tiempo real, conflictos de escritura, dependencia del medio).



```
PS C:\Users\ISAURO\Downloads\Sistemas Distribuidos\Practica2_ClienteServidor> python servidor/servidor.py
[SERVIDOR] Servicio activo y escuchando...

[SERVIDOR] Nuevo mensaje detectado: 'Hola Soy Isauro'
[SERVIDOR] Respuesta generada: 'PROCESADO: [HOLA SOY ISAURO] | Longitud: 15 caracteres'

[SERVIDOR] Nuevo mensaje detectado: 'PROBANDO MI SERVIDOR'
[SERVIDOR] Respuesta generada: 'PROCESADO: [PROBANDO MI SERVIDOR] | Longitud: 20 caracteres'

[SERVIDOR] Nuevo mensaje detectado: 'Al parecer funciona bien'
[SERVIDOR] Respuesta generada: 'PROCESADO: [AL PARECER FUNCIONA BIEN] | Longitud: 24 caracteres'

PS C:\Users\ISAURO\Downloads\Sistemas Distribuidos\Practica2_ClienteServidor> python cliente/cliente.py
python cliente/cliente.py
[CLIENTE] Escribiendo mensaje: 'Hola Soy Isauro'...
[CLIENTE] Esperando respuesta del servidor...
[CLIENTE] <<< Respuesta recibida: PROCESADO: [HOLA SOY ISAURO] | Longitud: 15 caracteres
PROBANDO MI SERVIDOR
[CLIENTE] Escribiendo mensaje: 'PROBANDO MI SERVIDOR'...
[CLIENTE] Esperando respuesta del servidor...
[CLIENTE] <<< Respuesta recibida: PROCESADO: [PROBANDO MI SERVIDOR] | Longitud: 20 caracteres
Ingrese un mensaje para enviar (o 'salir' para terminar): Al parecer funciona bien
[CLIENTE] Escribiendo mensaje: 'Al parecer funciona bien'...
[CLIENTE] Esperando respuesta del servidor...
[CLIENTE] <<< Respuesta recibida: PROCESADO: [AL PARECER FUNCIONA BIEN] | Longitud: 24 caracteres

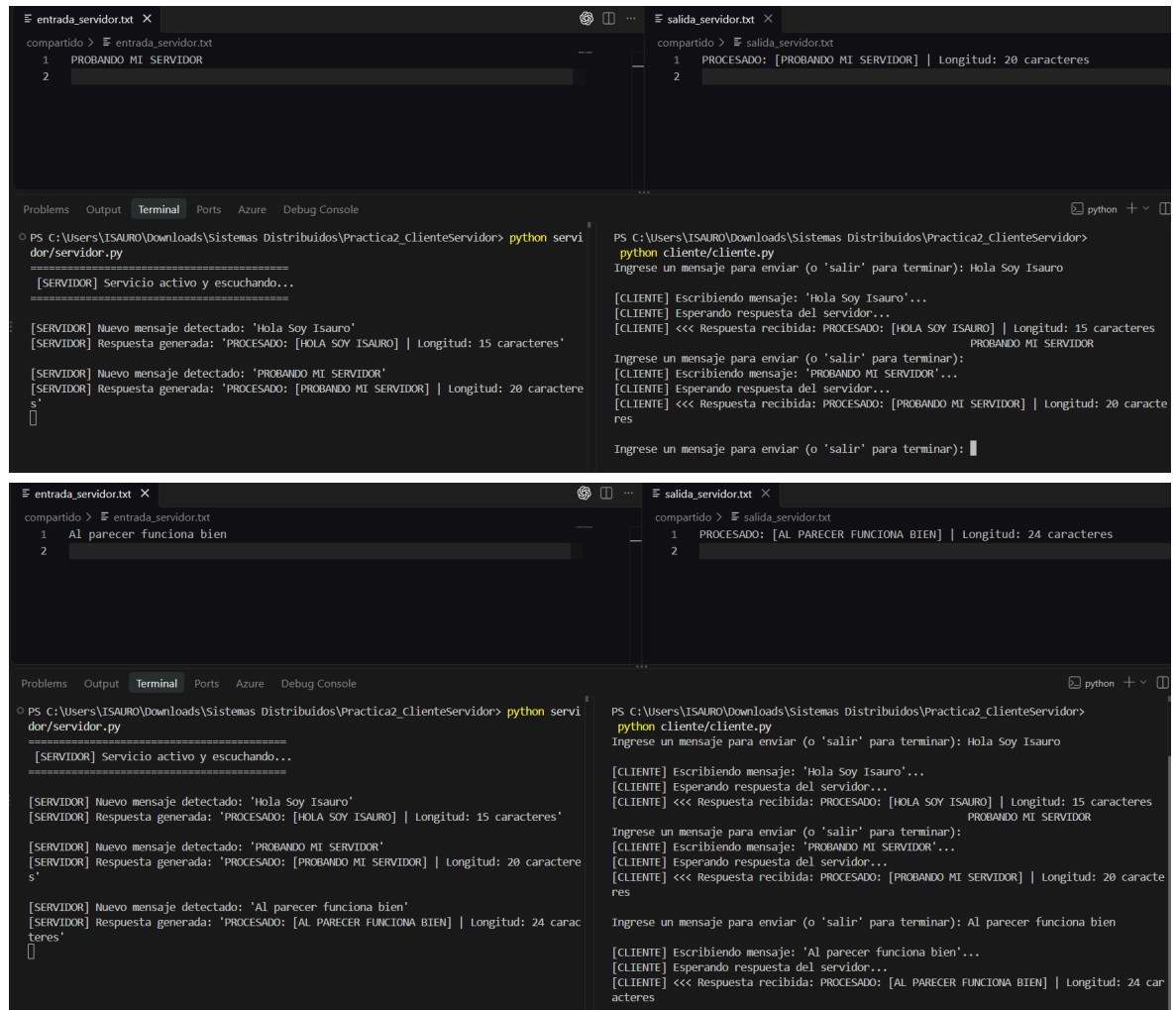
compartido > entrada_servidor.txt
1 Hola Soy Isauro
2

compartido > salida_servidor.txt
1 PROCESADO: [HOLA SOY ISAURO] | Longitud: 15 caracteres
2

PS C:\Users\ISAURO\Downloads\Sistemas Distribuidos\Practica2_ClienteServidor> python servidor/servidor.py
[SERVIDOR] Servicio activo y escuchando...

[SERVIDOR] Nuevo mensaje detectado: 'Hola Soy Isauro'
[SERVIDOR] Respuesta generada: 'PROCESADO: [HOLA SOY ISAURO] | Longitud: 15 caracteres'

PS C:\Users\ISAURO\Downloads\Sistemas Distribuidos\Practica2_ClienteServidor> python cliente/cliente.py
python cliente/cliente.py
Ingrese un mensaje para enviar (o 'salir' para terminar): Hola Soy Isauro
[CLIENTE] Escribiendo mensaje: 'Hola Soy Isauro'...
[CLIENTE] Esperando respuesta del servidor...
[CLIENTE] <<< Respuesta recibida: PROCESADO: [HOLA SOY ISAURO] | Longitud: 15 caracteres
Ingrese un mensaje para enviar (o 'salir' para terminar):
```



```

compartido > entrada_servidor.txt
1 PROBANDO MI SERVIDOR
2

compartido > salida_servidor.txt
1 PROCESADO: [PROBANDO MI SERVIDOR] | Longitud: 20 caracteres
2

PS C:\Users\ISAURO\Downloads\Sistemas Distribuidos\Practica2_clienteServidor> python servidor.py
[SERVIDOR] Servicio activo y escuchando...

[SERVIDOR] Nuevo mensaje detectado: 'Hola Soy Isauro'
[SERVIDOR] Respuesta generada: 'PROCESADO: [HOLA SOY ISAURO] | Longitud: 15 caracteres'

[SERVIDOR] Nuevo mensaje detectado: 'PROBANDO MI SERVIDOR'
[SERVIDOR] Respuesta generada: 'PROCESADO: [PROBANDO MI SERVIDOR] | Longitud: 20 caracteres'

PS C:\Users\ISAURO\Downloads\Sistemas Distribuidos\Practica2_clienteServidor> python cliente/cliente.py
Ingrese un mensaje para enviar (o 'salir' para terminar): Hola Soy Isauro

[CLIENTE] Escribiendo mensaje: 'Hola Soy Isauro'...
[CLIENTE] Esperando respuesta del servidor...
[CLIENTE] <<< Respuesta recibida: PROCESADO: [HOLA SOY ISAURO] | Longitud: 15 caracteres
PROBANDO MI SERVIDOR

Ingrese un mensaje para enviar (o 'salir' para terminar):
[CLIENTE] Escribiendo mensaje: 'PROBANDO MI SERVIDOR'...
[CLIENTE] Esperando respuesta del servidor...
[CLIENTE] <<< Respuesta recibida: PROCESADO: [PROBANDO MI SERVIDOR] | Longitud: 20 caracteres

Ingrese un mensaje para enviar (o 'salir' para terminar):

compartido > entrada_servidor.txt
1 Al parecer funciona bien
2

compartido > salida_servidor.txt
1 PROCESADO: [AL PARECER FUNCIONA BIEN] | Longitud: 24 caracteres
2

PS C:\Users\ISAURO\Downloads\Sistemas Distribuidos\Practica2_clienteServidor> python servidor.py
[SERVIDOR] Servicio activo y escuchando...

[SERVIDOR] Nuevo mensaje detectado: 'Hola Soy Isauro'
[SERVIDOR] Respuesta generada: 'PROCESADO: [HOLA SOY ISAURO] | Longitud: 15 caracteres'

[SERVIDOR] Nuevo mensaje detectado: 'PROBANDO MI SERVIDOR'
[SERVIDOR] Respuesta generada: 'PROCESADO: [PROBANDO MI SERVIDOR] | Longitud: 20 caracteres'

[SERVIDOR] Nuevo mensaje detectado: 'Al parecer funciona bien'
[SERVIDOR] Respuesta generada: 'PROCESADO: [AL PARECER FUNCIONA BIEN] | Longitud: 24 caracteres'

PS C:\Users\ISAURO\Downloads\Sistemas Distribuidos\Practica2_clienteServidor> python cliente/cliente.py
Ingrese un mensaje para enviar (o 'salir' para terminar): Hola Soy Isauro

[CLIENTE] Escribiendo mensaje: 'Hola Soy Isauro'...
[CLIENTE] Esperando respuesta del servidor...
[CLIENTE] <<< Respuesta recibida: PROCESADO: [HOLA SOY ISAURO] | Longitud: 15 caracteres
PROBANDO MI SERVIDOR

Ingrese un mensaje para enviar (o 'salir' para terminar):
[CLIENTE] Escribiendo mensaje: 'PROBANDO MI SERVIDOR'...
[CLIENTE] Esperando respuesta del servidor...
[CLIENTE] <<< Respuesta recibida: PROCESADO: [PROBANDO MI SERVIDOR] | Longitud: 20 caracteres

Ingrese un mensaje para enviar (o 'salir' para terminar): Al parecer funciona bien

[CLIENTE] Escribiendo mensaje: 'Al parecer funciona bien'...
[CLIENTE] Esperando respuesta del servidor...
[CLIENTE] <<< Respuesta recibida: PROCESADO: [AL PARECER FUNCIONA BIEN] | Longitud: 24 caracteres

```

7. Preguntas de Control:

¿Qué significa que cliente y servidor estén desacoplados?

Significa que no existe una dependencia directa ni un conocimiento interno entre ambos componentes del sistema.

El cliente ignora la dirección IP, el lenguaje de programación y la lógica interna con la que fue programado el servidor. A su vez, el servidor procesa peticiones sin conocer la identidad ni la ubicación física del cliente que las solicita. Toda la interacción se limita estrictamente a cumplir un contrato formal a través del medio compartido de comunicación.

¿Qué ventajas tiene desarrollar componentes por separado?

- Permite que diferentes equipos de desarrollo trabajen de forma paralela en el cliente y el servidor sin bloquearse.
- Facilita el mantenimiento, ya que se puede modificar la lógica interna de un módulo sin alterar el código del otro.
- Incrementa la reutilización de código, permitiendo conectar múltiples tipos de clientes a un mismo servicio central.
- Aísla los errores de implementación, haciendo que las pruebas unitarias y la detección de fallos sean mucho más simples.

¿Qué problemas pueden surgir al integrar sistemas independientes?

- Pueden surgir inconsistencias si alguno de los componentes rompe la estructura o el formato acordado para los datos.
- Se presentan errores de interpretación en los mensajes si no existe un manejo claro de excepciones y fallos. Aparecen cuellos de botella

inesperados cuando la velocidad de procesamiento del servidor no coincide con la del cliente.

- Surgen incompatibilidades del entorno, como diferencias en la codificación de caracteres o en los saltos de línea del SO.

¿Qué sucede si el servidor no está ejecutándose?

El cliente todavía puede depositar su mensaje en el archivo compartido debido al desacoplamiento temporal existente pero, al no haber un proceso activo que procese la información, la respuesta nunca será escrita en la memoria.

El cliente permanecerá en espera activa hasta que pase el tiempo límite configurado en su temporizador. Cuando se termine dicho tiempo máximo, el cliente finalizará su ciclo informando un error por tiempo de espera agotado.

¿Cómo se simula la comunicación en esta práctica?

Se simula una comunicación indirecta utilizando la memoria secundaria del equipo como si fuera un bus de mensajes. El archivo de entrada funciona como una cola de peticiones donde el cliente deposita sus mensajes de texto. El archivo de salida opera como una cola de respuestas que el servidor actualiza tras procesar cada solicitud. Ambos programas revisan continuamente el estado de estos archivos mediante consultas periódicas en un bucle activo.

¿Qué limitaciones tiene este modelo frente a uno real?

Genera un consumo ineficiente de memoria y CPU debido al sondeo constante que realizan ambos programas sobre el disco. Introduce una latencia alta en las respuestas, pues depende directamente de los tiempos de pausa del bucle.

No tiene un mecanismo de eventos o interrupciones que notifique los cambios de forma nativa e inmediata al sistema. No posee escalabilidad distribuida directa, quedando limitado a ejecutarse dentro de un mismo sistema de archivos local.

¿Cómo se podría mejorar la sincronización?

Sustituyendo la consulta periódica por un esquema basado en eventos de modificación del sistema de archivos. Implementando mecanismos de bloqueo de archivos para evitar conflictos cuando ambos procesos accedan al tiempo. Evolucionando el diseño hacia *sockets* de red TCP/UDP para transferir mensajes directamente en memoria RAM. Adoptando intermediarios de mensajería (*message brokers*) como RabbitMQ o Redis para gestionar las colas de peticiones.

¿Qué pasaría si múltiples clientes usan el sistema?

Se producirían condiciones de carrera al intentar escribir varios clientes de manera simultánea en el mismo archivo. El sistema operativo arrojaría errores de permisos denegados al detectar accesos concurrentes sobre el recurso. Los mensajes de los usuarios se sobrescribirían entre sí, corrompiendo la información almacenada en la entrada. Las respuestas escritas por el servidor se mezclarían, enviando resultados cruzados o erróneos a los clientes.

8. Evaluación

NO APLICA

9. Bibliografía

Semana 2. Guaman, Jose. 2026

10. Elaboración y Aprobación

Elaborado por	José Guamán Docente	Validar Únicamente en el Firmado Electrónico con Fimnec JOSE OSWALDO GUAMAN QUINCHE 
Ejecutado Por	Rene Rivera Técnico Docente	Validar Únicamente en el Firmado Electrónico con Fimnec RAMIRO RENE RIVERA GUAMAN 
Aprobado por	Edison L Coronel Romero Director de Carrera	Firmado Electrónico con Fimnec  Validar Únicamente con Fimnec EDISON L CORONEL ROMERO

◆ Declaración del uso de IA

Declaró que, para la elaboración de esta tarea, hice uso de una herramienta de Inteligencia Artificial (Notebook) únicamente como apoyo para mejorar la redacción y síntesis de ideas.

El uso de la IA se realizó respetando el Contrato de uso ético y responsable de la Inteligencia Artificial, comprometiéndose a:

- No copiar directamente sin comprensión de los textos generados.
- Revisar, adaptar y redactar las ideas en nuestras propias palabras.
- Mantener un aprendizaje activo y crítico en el desarrollo del trabajo.

Prompt

Actúa como un profesor experto en Sistemas Distribuidos. Necesito que me guíes paso a paso en la implementación de una práctica de comunicación indirecta mediante archivos compartidos en Python, guíame en:

1. La verificación de la estructura de carpetas necesaria antes de ejecutar.
2. Cómo ejecutar ambos scripts en terminales independientes para probar el flujo con al menos 3 mensajes.



Universidad
Nacional
de Loja

FEIRNNR - Carrera de
Computación

3. Explicación de cómo este ejercicio demuestra el desacoplamiento espacial, temporal e implementativo.
4. Identificación de los puntos donde pueden ocurrir condiciones de carrera (Race Conditions) y errores de permisos de archivo. Dame explicaciones claras y concisas, enfocadas en la arquitectura del sistema.